
Task W04: Linked Lists

Week 4 (These boots were made for walking.)

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX), but I have personally reviewed and worked through every problem in it, tracing through the algorithms by hand, running code where relevant, and reasoning through the proofs myself, so it reflects exactly what I intend to ask you. If something behaves in a way you do not expect, that is not evidence of an oversight on my part; work through it carefully rather than assuming it is a mistake. If, after real effort, you still believe something is genuinely wrong, find me in the Sandbox. It may be an interesting conversation.

This assignment is conceptual, not a coding assignment, though a couple of questions reference short code or pseudocode. The goal is to see whether you have actually built a working mental model of singly linked, doubly linked, and circular linked lists, not whether you can restate a definition. Answer in your own words, and be specific: a vague restatement of the question is not the same as an answer, and a few of these questions say so directly.

A Quick Preview: What “ $O(1)$ ” and “ $O(n)$ ” Mean

You will see this notation properly later in the semester, but Question 1 below uses the phrase “constant time,” which is exactly what $O(1)$ means, so here is just enough to make sense of it.

When we say an operation is $O(1)$, we mean its cost stays the same no matter how big the list is. `addFirst` is $O(1)$: whether the list has 3 nodes or 3 million, adding to the front touches exactly one node and reassigns exactly one reference. The size of the list never enters into it.

When we say an operation is $O(n)$, we mean its cost grows in direct proportion to the number of items, conventionally called n . `addLast` without a tail reference is $O(n)$: to add one node, you have to walk past every existing node first, so twice the list means twice the work.

That is the whole preview. The real version of this idea, formal Big-O notation, where it comes from, and why we care, is coming later. For now, “ $O(1)$ ” just means “constant effort regardless of size,” and “ $O(n)$ ” means “effort scales with size.”

1 Deletion Without a Predecessor

In a singly linked list, if you are handed a reference to some node in the middle of the list (not the head), you cannot delete that node in true constant time the way you could in a doubly linked list. Explain why, and describe what the singly linked list is missing that would let it do this.

2 Choosing the Wrong Structure

Doubly linked lists let you walk backward, but that is not automatically worth having. Describe a realistic situation where choosing a doubly linked list over a singly linked one would be the wrong call.

To earn full credit, your answer needs all three of the following:

- A specific, named program or use case, not a general description
- The specific capability of a doubly linked list that this use case never exercises
- The specific cost of carrying that unused capability anyway, stated in concrete terms (memory, code complexity, or maintenance risk)

“It uses more memory” on its own, with no scenario attached, will not earn credit.

3 Reversing a Linked List

The pseudocode below reverses a singly linked list in place, so that $L.head$ ends up pointing at what used to be the last node. (A note on notation: this pseudocode uses NIL where we have used null in class; they refer to the same thing.)

```

REVERSE-LIST( $L$ )
1   $prev \leftarrow \text{NIL}$ 
2   $current \leftarrow L.head$ 
3  while  $current \neq \text{NIL}$ 
4     $next \leftarrow current.next$ 
5     $prev \leftarrow current$ 
6     $current.next \leftarrow prev$ 
7     $current \leftarrow next$ 
8   $L.head \leftarrow prev$ 

```

- (a) Trace this algorithm by hand on the list $K \rightarrow F \rightarrow Q \rightarrow M \rightarrow \text{NIL}$. Show the values of $prev$, $current$, and $next$, and the state of every node's $next$ field, after each pass through the loop. **Your hand trace itself must be included in your submission, not just your final answer.** A suggested layout is below; you may copy it and fill in the blanks, or use your own format as long as it shows the same information.

Table 1: A suggested layout for your trace. Copy this structure and fill in one row per iteration.

Iteration	prev	current	next	K.next	F.next	Q.next	M.next
start	NIL	K	—	F	Q	M	NIL
1							
2							
3							
4							

- (b) Using your completed trace, write out, in order, the sequence of values you get by walking the list starting from $L.head$ once the algorithm finishes.

4 The Dummy Node

A dummy node holds no real data, and it sits in memory for the entire life of the list, whether the list has zero items or a million.

-
- (a) Explain specifically what changes about how insertion and deletion are implemented when a list uses a dummy node, compared to one that does not.

To earn full credit, your answer needs both of the following:

- A specific check or condition that a dummy-less implementation needs and a dummy-node implementation does not
- An explanation of why the dummy node removes the need for that specific condition, not just that it does

“It makes insertion and deletion simpler” on its own, with no specific condition named, will not earn credit.

- (b) Describe a situation where you would deliberately choose not to use a dummy node, even though it would simplify your code.

To earn full credit, your answer needs both of the following:

- A specific, named program or use case, not a general description
- The specific cost of the dummy node in that situation, stated in concrete terms

“It uses more memory” on its own, with no scenario attached, will not earn credit.

5 Challenge: When next Points Backward

This question is optional and will not affect your grade in any way. It is here for anyone who wants to try something harder. If you attempt it, please still turn in your work, even though it will not be scored.

A singly linked list is supposed to end in NIL. Suppose a bug elsewhere in a program has caused some node’s *next* field to be reassigned so that it now points back to an earlier node in the list, instead of continuing forward or ending in NIL. You do not know whether this has happened to a given list. You are not allowed to find out by storing visited node references in a set, array, or any structure whose size depends on the list’s length. You may use only a fixed number of pointer variables, decided in advance and not dependent on the list’s length, the same handful of variables would work whether the list has ten nodes or ten million. Use as few as you reasonably can; fewer variables are usually easier to reason about and prove things about.

- (a) Describe an algorithm that determines whether a list has been altered this way, under the constraint above.
- (b) Your algorithm almost certainly involves two pointers moving through the list at different speeds. Prove that if the list has been altered this way, these two pointers are guaranteed to eventually point to the same node. A vague appeal to “the fast one laps the slow one” is not a proof. Reason about the distance between the two pointers at each step, and show that this distance must reach a specific value, not just that it “probably will.”
- (c) Once your two pointers meet, describe how you would identify the exact earlier node that a *next* field now incorrectly points back to, not merely that this has happened, again using only a fixed number of pointer variables. Prove that your method finds the correct node.